

```
GNU nano 8.7 bankers.c Modified
#include <stdio.h>

#define P 5 // Number of processes
#define R 4 // Number of resources

int main() {

    int allocation[P][R] = {
        {0,0,1,4}, // Google Drive
        {0,6,3,2}, // Firefox
        {0,0,1,2}, // Word Processor
        {1,0,0,0}, // Excel
        {1,3,5,4} // PowerPoint
    };

    int max[P][R] = {
        {0,6,5,6},
        {0,6,5,2},
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6}
    };

    int available[R] = {1,6,2,0};

    int need[P][R];
    int finish[P] = {0};
    int safeSeq[P];
    int work[R];

    printf("\nNeed Matrix:\n");

    // Calculate Need Matrix
    for(int i=0;i<P;i++){
        for(int j=0;j<R;j++){
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    AG Help      AG Write Out  AF Where Is   AK Cut        AT Execute    AC Location   M-U Undo      M-A Set Mark  M-J To Bracket
    AX Exit      AR Read File  AL Replace    AU Paste      AJ Justify    AV Go To Line M-E Redo      M-G Copy      AB Where Was
```

```
GNU nano 8.7 bankers.c Modified
}

// Copy available to work
for(int i=0;i<R;i++)
    work[i] = available[i];

int count = 0;

while(count < P){
    int found = 0;

    for(int i=0;i<P;i++){
        if(!finish[i]){
            int j;
            for(j=0;j<R;j++){
                if(need[i][j] > work[j])
                    break;
            }

            if(j == R){
                for(int k=0;k<R;k++)
                    work[k] += allocation[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(!found){
        printf("\nSystem is NOT in safe state!\n");
        return 0;
    }
}

printf("\nSystem is in SAFE state.\nSafe Sequence:\n");

char *processName[P] = {
    "GoogleDrive",
    "Firefox",

```

```
GNU nano 8.7 bankers.c Modified
int j;
for(j=0;j<R;j++){
    if(need[i][j] > work[j])
        break;
}

if(j == R){
    for(int k=0;k<R;k++){
        work[k] += allocation[i][k];

        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}

if(!found){
    printf("\nSystem is NOT in safe state!\n");
    return 0;
}

printf("\nSystem is in SAFE state.\nSafe Sequence:\n");

char *processName[P] = {
    "GoogleDrive",
    "Firefox",
    "WordProcessor",
    "Excel",
    "PowerPoint"
};

for(int i=0;i<P;i++){
    printf("%s -> ", processName[safeSeq[i]]);
}

printf("END\n");

return 0;
}
```

```
athar@THOR:MSYS ~
$ nano bankers.c

athar@THOR:MSYS ~
$ gcc bankers.c -o bankers

athar@THOR:MSYS ~
$ ./bankers

Need Matrix:
0 6 4 2
0 0 2 0
0 0 0 0
0 7 5 0
1 0 0 2

System is in SAFE state.
Safe Sequence:
Firefox -> WordProcessor -> Excel -> PowerPoint -> GoogleDrive -> END

athar@THOR:MSYS ~
$
```